

Universidade Federal do Espírito Santo
Departamento de Informática

Trabalho Prático

Jogo da Void

Programação II - 2026/2

31 de outubro de 2026

Objetivo

O objetivo deste trabalho é colocar em prática as habilidades de programação na linguagem C adquiridas ao longo do curso. O trabalho foi elaborado para você exercitar diversos conceitos, principalmente a construção de TADs opacos e genéricos, alocação e manipulação de memória de maneira dinâmica. Como consta no plano de ensino da disciplina, o trabalho será realizado em duas etapas (a primeira realizada em casa e a segunda em sala de aula), sendo esta a primeira etapa. O somatório de ambas as etapas **equivale a 50% da nota final do curso**.

Introdução

Jogos de tabuleiro estão entre as formas mais antigas de entretenimento. Em muitos deles, os jogadores percorrem uma trilha lançando dados e, a cada casa alcançada, algo acontece: um tesouro é encontrado, um monstro precisa ser enfrentado, o jogador avança ou é obrigado a recuar. Versões digitais desses jogos precisam representar o tabuleiro, controlar os turnos, aplicar as regras de cada casa e manter o estado de cada jogador ao longo da partida.

Neste trabalho, você implementará o **Jogo da Void**, um protótipo simplificado de jogo de tabuleiro digital em que N jogadores disputam uma corrida por uma trilha linear repleta de desafios.

Descrição geral do sistema

O Jogo da Void deve simular uma partida de um jogo de tabuleiro. Para isso, considere:

- O tabuleiro é um mapa linear (uma trilha) composto por células, delimitado por uma célula de Início e uma célula de Fim
- O mapa pode variar de uma partida para outra, tanto na quantidade quanto nos tipos de células
- Cada célula possui uma ação, que é disparada quando um jogador para nela
- Os jogadores jogam em turnos, seguindo a ordem de cadastro, e avançam de acordo com o valor obtido em um dado
- Cada jogador possui força, experiência (XP) e ouro
- Cada célula comporta apenas um jogador: quem chega a uma célula ocupada precisa disputá-la
- Vence o primeiro jogador a alcançar o Fim do mapa
- O sistema deve exibir o estado da partida, um ranking e um relatório geral quando solicitado.

Funcionalidades essenciais:

1. Cadastro do Mapa
2. Cadastro de Jogadores
3. Execução das rolagens (simulação da partida)
4. Outras ações

Cadastro do Mapa

A primeira funcionalidade do Jogo do Void é o cadastro do mapa da partida. O mapa é informado via entrada padrão entre as linhas `INICIO MAPA` e `FIM MAPA`. Cada linha entre elas descreve uma célula, na ordem em que as células aparecem na trilha.

A quantidade de células não é informada na entrada e deve ser inferida durante a leitura. Portanto, o mapa deve ser armazenado em uma estrutura alocada dinamicamente, capaz de crescer à medida que novas células são lidas.

As células informadas são numeradas a partir de 1. As células de Início e de Fim não são informadas na entrada, existem em todo mapa e não possuem ação. Em um mapa com N

células informadas, o Início corresponde à casa (i.e., célula) 0 e o Fim corresponde à casa N + 1.

Cada célula possui um tipo, que determina a ação executada quando um jogador para nela. Os tipos de célula são:

- **VAZIA**: nada acontece.
- **OURO <quantidade>**: o jogador coleta a quantidade de ouro indicada.
- **MONSTRO <nome> <força> <xp>**: o jogador enfrenta o monstro. Se a força do jogador for maior ou igual à força do monstro, o monstro é derrotado e o jogador ganha a XP indicada. Caso contrário, o jogador perde o confronto e recua (ver regra de recuo).
- **ITEM <nome> <preço> <bônus>**: a célula vende um item. Se o jogador possuir ouro suficiente, ele compra o item automaticamente (não há opção de recusar): o preço é descontado do seu ouro e o bônus é somado à sua força. Caso contrário, nada acontece.
- **AVANCA <n>**: o jogador avança n casas.
- **VOLTA <n>**: o jogador volta n casas. Se o recuo ultrapassar o Início, o jogador para no Início.

Nomes de monstros e itens são strings com no máximo 20 caracteres e sem espaços. Todos os valores numéricos são inteiros positivos. As células são **permanentes**: o ouro de uma célula pode ser coletado várias vezes, assim como um monstro pode ser enfrentado por qualquer jogador que pare em sua célula, mesmo que já tenha sido derrotado antes.

Um exemplo de cadastro de mapa é mostrado a seguir:

```
INICIO MAPA (início do cadastro do mapa)
OURO 10 (casa 1)
MONSTRO GOBLIN 3 10 (casa 2: nome, força e xp do monstro)
AVANCA 2 (casa 3)
VAZIA (casa 4)
ITEM ESPADA 10 2 (casa 5: nome, preço e bônus de força)
MONSTRO ORC 6 25 (casa 6)
VOLTA 3 (casa 7)
OURO 20 (casa 8)
VAZIA (casa 9)
MONSTRO DRAGAO 9 50 (casa 10)
OURO 5 (casa 11)
VAZIA (casa 12)
FIM MAPA (fim do cadastro do mapa)
```

Neste exemplo, o mapa possui 12 células. O Início é a casa 0 e o Fim é a casa 13.

Ações genéricas

O Jogo da Void foi pensado para ser facilmente ampliado com novos tipos de célula. Por isso, **a ação de uma célula deve ser representada por um TAD genérico**, que armazena os dados específicos de cada tipo por meio de um ponteiro `void*` e as operações sobre esses dados por meio de ponteiros de função (executar, imprimir e liberar). O código responsável por conduzir a partida não deve conhecer os tipos concretos de ação: ele apenas solicita que a ação da célula seja executada sobre o jogador.

Recomenda-se que a execução de uma ação atue apenas sobre o jogador e informe o deslocamento resultante (por exemplo, 0 para OURO, +2 para AVANCA 2 e -3 para VOLTA 3), deixando a movimentação no mapa a cargo do módulo que conduz a partida. A interface exata está definida nos templates de código disponibilizados.

Cadastro de Jogadores

O cadastro de jogadores é realizado via entrada padrão e é representado pelo caractere J. Cada jogador possui pelo menos as seguintes informações:

- Nome (string com no máximo 50 caracteres)
- Cor (string com no máximo 15 caracteres, ex: VERMELHO, VERDE, AMARELO)
- Força inicial (inteiro)

Todo jogador inicia a partida no Início do mapa (casa 0), com 0 de XP e 0 de ouro. O nome é o identificador único de um jogador. O sistema não deve permitir o cadastro de dois jogadores com o mesmo nome. Caso seja realizada uma tentativa de cadastro com um nome já existente, o sistema deve ignorar e não realizar o cadastro. A quantidade de jogadores não é conhecida previamente.

Um exemplo de cadastro de jogadores é ilustrado a seguir:

```
J (caractere indicando cadastro de um jogador)
ANA (nome)
VERMELHO (cor)
5 (força inicial)
J (caractere indicando cadastro de um jogador)
BRUNO (nome)
VERDE (cor)
5 (força inicial)
J (caractere indicando cadastro de um jogador)
CARLA (nome)
AMARELO (cor)
4 (força inicial)
```

A sequência de leitura sempre será: mapa, jogadores e rolagens. Sendo assim, o mapa e todos os jogadores serão cadastrados antes da primeira rolagem.

Execução das rolagens

A partida avança por meio do comando `ROLAGEM <n>`, em que n é a quantidade de jogadas a serem executadas. Os jogadores jogam em turnos, seguindo a ordem de cadastro: o primeiro jogador cadastrado faz a primeira jogada, depois o segundo e assim por diante. Após o último jogador, a vez retorna ao primeiro. A ordem dos turnos deve ser controlada de forma rotativa: o primeiro jogador realiza a rolagem, depois o segundo, o terceiro, até chegar no último. Quando isso ocorrer, o turno volta para o primeiro jogador novamente. Isso é realizado até que o jogo seja finalizado.

Por exemplo, em uma partida com 3 jogadores, o comando a seguir executa 4 jogadas: uma para o jogador 1, uma para o jogador 2, uma para o jogador 3 e, novamente, uma para o jogador 1. Uma nova rolagem continua a partir do jogador seguinte.

```
ROLAGEM 4 (executa 4 jogadas)
```

Se a partida terminar durante uma rolagem, as jogadas restantes não são executadas. Por exemplo, se for solicitado `ROLAGEM 50` e um jogador alcançar o Fim na 10ª jogada, a partida é encerrada nesse momento. Rolagens solicitadas após o fim da partida devem ser ignoradas.

O dado

O valor do dado é obtido pela função `RolaDado()`, fornecida no template de código (`dado.h`). Ela retorna um inteiro entre 1 e 4 seguindo uma sequência pseudoaleatória pré-determinada, o que garante que a mesma entrada sempre produza a mesma partida. **A função deve ser chamada exatamente uma vez por jogada**, e somente quando um jogador for lançar o dado. Chamadas adicionais alteram a sequência de valores e, consequentemente, toda a partida. Os arquivos de implementação do dado não devem ser modificados.

Regras de uma jogada

Cada jogada segue os seguintes passos:

1. O jogador da vez lança o dado e se desloca a quantidade de casas obtida, a partir da sua posição atual.
2. Se a casa de destino for o Fim ou ultrapassá-lo, o jogador alcança o Fim, vence a partida e o jogo é encerrado. Não é necessário obter o valor exato para alcançar o Fim.
3. Se a casa de destino estiver ocupada por outro jogador, ocorre uma disputa. Se o jogador da vez perder a disputa, ele recua (ver regra de recuo) e sua jogada termina. Se vencer, o ocupante é quem recua.
4. Se o jogador da vez permanecer na casa de destino, a ação da célula é executada.
5. Se a ação da célula alcançada resultar em um deslocamento (AVANCA ou VOLTA), o jogador é movido. **A ação da nova célula alcançada por esse deslocamento não é executada**, mas, se ela estiver ocupada, ocorre uma disputa normalmente. Se o deslocamento alcançar ou ultrapassar o Fim, o jogador vence a partida.
6. Após o término da jogada, o jogador retorna ao final da fila de turnos.

Disputa territorial

Cada célula comporta apenas um jogador, com exceção do Início, que comporta todos. Quando um jogador chega a uma célula ocupada, os dois disputam a célula:

- Vence quem possuir mais força
- Em caso de empate na força, vence quem possuir mais XP
- Em caso de novo empate, o ocupante permanece na célula

O perdedor recua de acordo com a regra de recuo e mantém sua força. Somente o vencedor permanece na célula disputada.

Regra de recuo

Sempre que um jogador precisar recuar (por perder uma disputa ou o confronto com um monstro), ele volta para a casa imediatamente anterior à casa em que estava. Se essa casa estiver ocupada, ele continua recuando até encontrar a primeira casa livre ou até chegar ao Início. O recuo nunca dispara a ação da célula alcançada nem uma nova disputa.

Exemplo comentado

Considere o mapa e os jogadores dos exemplos anteriores, e suponha que a função `RolaDado()` retorne os valores 1, 3, 2 e 4 nas quatro primeiras chamadas. O comando `ROLAGEM 4` produz:

- ANA tira 1, para na casa 1 (OURO 10) e coleta 10 de ouro.
- BRUNO tira 3 e para na casa 3 (AVANCA 2). Ele avança para a casa 5, mas a ação da casa 5 (ITEM) não é executada.
- CARLA tira 2 e para na casa 2 (MONSTRO GOBLIN). Como sua força (4) é maior ou igual à do monstro (3), ela o derrota e ganha 10 de XP.
- ANA tira 4 e chega à casa 5, ocupada por BRUNO. Ambos possuem força 5 e XP 0, então o ocupante permanece. ANA recua para a casa 4, que está livre.

Outras ações disponíveis no sistema

O sistema deve permitir a execução de outras solicitações baseadas no mapa, nos jogadores cadastrados e no andamento da partida. Essas solicitações podem ser realizadas a qualquer momento após os cadastros e são descritas a seguir.

Estado da partida

O estado atual da partida pode ser consultado da seguinte forma:

```
E (caractere indicando que alguma solicitação será executada)
ESTADO (tipo solicitado)
```

Uma vez solicitado, o estado deve ser exibido na saída padrão. Primeiro, cada célula do mapa é exibida em ordem, com o nome do jogador que a ocupa, se houver. Na linha do Início são listados todos os jogadores que estão nele, na ordem de cadastro, e na linha do Fim é exibido o vencedor, se houver. Em seguida, são exibidos as informações de cada jogador, na ordem de cadastro, e a situação da partida (`EM ANDAMENTO` ou `VENCEDOR <nome>`). A saída para o exemplo comentado é mostrada a seguir:

```
========== ESTADO ============
[INICIO]
[1] OURO 10
[2] MONSTRO GOBLIN (F:3 XP:10) <- CARLA
[3] AVANCA 2
[4] VAZIA <- ANA
[5] ITEM ESPADA (P:10 F:+2) <- BRUNO
[6] MONSTRO ORC (F:6 XP:25)
[7] VOLTA 3
[8] OURO 20
[9] VAZIA
[10] MONSTRO DRAGAO (F:9 XP:50)
[11] OURO 5
[12] VAZIA
[FIM]
------------------------------
- ANA: casa 4 | forca 5 | xp 0 | ouro 10
- BRUNO: casa 5 | forca 5 | xp 0 | ouro 0
- CARLA: casa 2 | forca 4 | xp 10 | ouro 0
- Situacao: EM ANDAMENTO
==============================
```

Observe que cada tipo de célula possui sua própria forma de exibição. Como as células são representadas por ações genéricas, a exibição também deve ser feita pela função de impressão de cada tipo de ação.

Ranking

O ranking da partida pode ser solicitado da seguinte forma:

```
E (caractere indicando que alguma solicitação será executada)
RANKING (tipo solicitado)
```

Os jogadores devem ser exibidos em ordem decrescente de posição no mapa, ou seja, quem estiver mais próximo do Fim aparece no topo (naturalmente, o vencedor, se houver, ocupa a primeira posição). Em caso de empate na posição, vence quem possuir mais força. Persistindo o empate, vence quem possuir mais XP e, por fim, os jogadores devem aparecer em ordem alfabética.

Supondo que, após o exemplo comentado, a partida continue com o comando `ROLAGEM 50` e termine com a vitória de CARLA, o ranking seria:

```
========== RANKING ===========
1. CARLA (AMARELO) | casa 13 | forca 4 | xp 10
2. BRUNO (VERDE) | casa 12 | forca 5 | xp 0
3. ANA (VERMELHO) | casa 9 | forca 5 | xp 0
==============================
```

Relatório geral

Por fim, o sistema deve ser capaz de gerar um relatório geral da partida. Essa solicitação é realizada da seguinte forma:

```
E (caractere indicando que alguma solicitação será executada)
RELATORIO (ação solicitada)
```

O relatório deve exibir as seguintes informações:

- Quantidade de células do mapa (sem contar Início e Fim)
- Quantidade de jogadores cadastrados
- Quantidade total de jogadas executadas
- Quantidade total de disputas ocorridas
- Quantidade total de monstros derrotados
- Situação da partida
- Para cada jogador, na ordem de cadastro: quantidade de jogadas, disputas vencidas, força, XP e ouro

Essas informações devem ser exibidas na tela da seguinte forma (continuando o exemplo anterior):

```
========= RELATORIO ===========
- Celulas do mapa: 12
- Jogadores: 3
- Jogadas totais: 12
- Disputas: 3
- Monstros derrotados: 1
- Situacao: VENCEDOR CARLA
------------------------------
- ANA: jogadas 4 | disputas vencidas 1 | forca 5 | xp 0 | ouro 10
- BRUNO: jogadas 4 | disputas vencidas 2 | forca 5 | xp 0 | ouro 20
- CARLA: jogadas 4 | disputas vencidas 0 | forca 4 | xp 10 | ouro 0
==============================
```

Importante: o estado, o ranking e o relatório refletem as informações no momento em que foram solicitados.

Exemplo de entrada completa

Um arquivo de entrada completo segue a estrutura a seguir. O programa deve ser encerrado quando o caractere F for lido.

```
INICIO MAPA
OURO 10
MONSTRO GOBLIN 3 10
... (demais células)
FIM MAPA
J
ANA
VERMELHO
5
... (demais jogadores)
ROLAGEM 4
E
ESTADO
ROLAGEM 50
E
RANKING
E
RELATORIO
F (finaliza o programa)
```

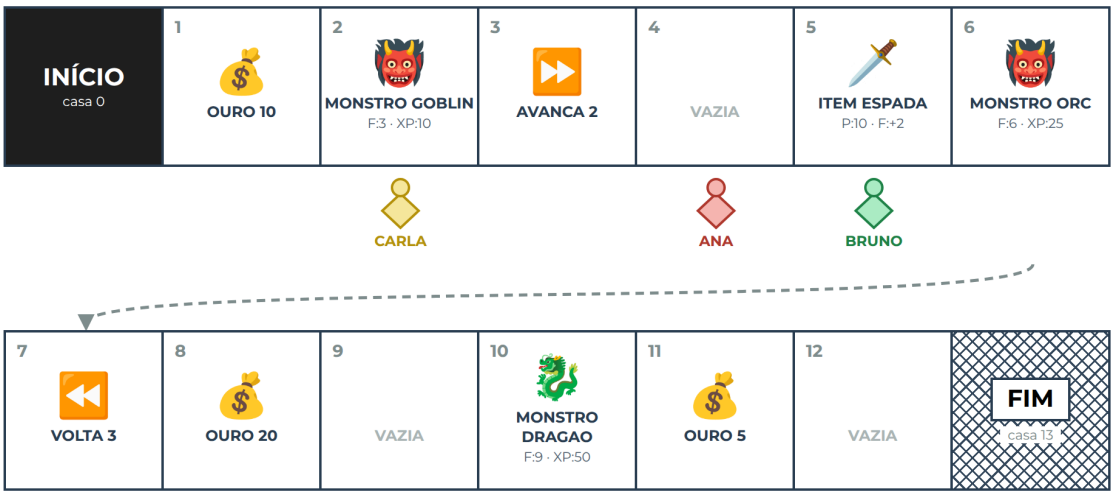


Figura 1: ilustração do tabuleiro do Jogo da Void

Informações gerais

Alguns arquivos de entrada e respectivos arquivos de saída serão fornecidos para o aluno. O aluno deverá utilizar tais arquivos para testes durante a implementação do trabalho. É de responsabilidade do aluno criar novos arquivos para testar outras possibilidades do programa e garantir seu correto funcionamento. O trabalho será corrigido usando, além dos arquivos dados, outros arquivos (específicos para a correção e não disponibilizados para os alunos) seguindo a formatação descrita neste documento. Em caso de dúvida, entre em contato com o professor e/ou monitor.

O uso de arquivos com formatação diferente poderá acarretar em incompatibilidade durante a correção do trabalho e consequentemente na impossibilidade de correção do mesmo (sendo atribuído a nota zero). Portanto, siga estritamente o formato estabelecido.

Implementação e verificação de resultados

A implementação deverá seguir o template de código disponibilizado no repositório do trabalho dentro da organização da disciplina no Github:

- Link para o repositório: https://github.com/prog-II-ufes/2026-2-template-TP-etapa-1

A utilização do template segue os mesmos princípios dos exercícios disponibilizados ao longo da disciplina. Você deve utilizar os arquivos .h disponibilizados e, obrigatoriamente,

implementar todas as funcionalidades requisitadas para a construção destes TADs. Todavia, vocês são livres para implementar outros TADs e/ou bibliotecas que julgarem necessárias. Além disso, você já possui acesso ao script de correção automática de exercícios e trabalhos. Ele será utilizado para corrigir o seu trabalho. Desta forma, é extremamente importante que você avalie seu trabalho utilizando o script e os casos de teste disponibilizados.

Para fins de depuração, você também pode utilizar softwares de comparação de arquivos, como diff e o Meld (alternativa gráfica do diff). Diferenças na formatação poderão impossibilitar a comparação e consequentemente impossibilitar a correção do trabalho. O programa será considerado correto se gerar a saída esperada idêntica à fornecida com os casos de teste.

Pontuação e Processo de Correção

A primeira etapa do trabalho será pontuada de acordo com sua implementação e seguindo os pontos apresentados na tabela a seguir.

Item	Descrição	Peso
jogador.h	TAD que representa um jogador	15%
mapa.h	TAD que representa um mapa com a quantidade necessária de células para executar o jogo.	5%
celula.h	TAD genérico relacionado à célula do mapa.	5%
acao_monstro.h	TAD relacionado à ação do tipo MONSTRO.	5%
main.c + restante	Função principal do programa + restante do código necessário.	70%

A pontuação da tabela será utilizada para calcular a nota para cada caso de teste. A pontuação final será a média de pontos considerando todos os casos. Haverá casos de teste visíveis e ocultos (que serão divulgados durante a segunda etapa do trabalho em sala).

A pontuação obtida de acordo com a tabela anterior será utilizada para calcular a nota final do seu trabalho. O gerenciamento de memória e modularização são itens obrigatórios. Sendo assim, não basta apenas acertar a saída; é necessário seguir os templates de código disponibilizados e gerenciar corretamente o uso da memória. Sendo assim, é de suma importância que seu trabalho siga à risca os templates e gerencie corretamente a memória (**lembre-se, esses são os principais objetivos desta disciplina**). Todo esse processo de

pontuação será realizado por meio do script de correção que vocês já possuem acesso. Ver a documentação sobre o uso do script caso necessário. Observe que a compilação da pasta completo (dentro de resultados) no script não é utilizada para computar a pontuação.

Processo de correção do trabalho: como consta no plano de ensino da disciplina, o trabalho prático é dividido em duas etapas. A primeira etapa é o desenvolvimento do projeto seguindo as regras e descrições apresentadas neste documento. A segunda etapa consiste na correção, atualização e/ou incremento de funcionalidades implementadas na etapa anterior. Esta última etapa é realizada em sala de aula de acordo com cronograma da disciplina. A nota final do trabalho é a média ponderada entre as duas etapas, de acordo com:

Nota final = 0.1 * Etapa 1 + 0.9 * Etapa 2

É importante ressaltar que ao iniciar a etapa 2 vocês terão acesso à nota e aos casos de teste ocultos da etapa 1. Portanto, vocês poderão corrigir possíveis erros cometidos durante o desenvolvimento da etapa 1.

Prazo e submissão

Todas as informações referentes a prazo de entrega e forma de submissão estão disponíveis na atividade **Trabalho Prático - Etapa 1** no classroom da disciplina.

Regras gerais

Para todas as etapas do trabalho, considere as seguintes regras:

- Trabalhos entregues após o prazo não serão corrigidos (recebendo a nota zero)
- O trabalho deverá ser feito **individualmente** e pelo próprio aluno, isto é, o aluno deverá necessariamente conhecer e dominar todos os trechos de código criados. Cada aluno deverá trabalhar independente dos outros, não sendo permitido a cópia ou compartilhamento de código.
    - Haverá verificação automatizada de plágio. Trabalhos identificados como iguais, em termos de programação (por exemplo, mudar nomes de variáveis e funções, entre outros, não faz dois trabalhos serem diferentes), serão penalizados com a nota zero e poderão ser submetidos para penalidades adicionais em instâncias superiores. Isso também inclui a pessoa que forneceu o trabalho, sendo, portanto, de sua obrigação a proteção de seu trabalho

contra cópias ilícitas. Proteja seu trabalho e não esqueça cópias do seu código nas máquinas de uso comum.

- Nenhuma entrada dos dados nos casos de teste vai possuir acentos ou caracteres especiais. Além disso, estarão sempre em caixa alta (ex: JOAO).
- Todos os arquivos de entrada vão seguir sempre o mesmo padrão.
- Todas as entradas de dados serão fornecidas de acordo com a descrição. Por exemplo, todas as células do mapa estarão entre as linhas INICIO MAPA e FIM MAPA, e todos os cadastros ocorrerão antes da primeira rolagem.
- Todos os valores numéricos do trabalho (força, XP, ouro, preço, bônus e deslocamentos) são inteiros.
- Para dados multidimensionais no qual o tamanho é desconhecido (como o mapa e o conjunto de jogadores), é **obrigatório** o uso de alocação dinâmica de memória. Liberar a memória alocada é de sua responsabilidade.
    - Caso você tente burlar essa regra, você será penalizado
- O valor do dado deve ser obtido exclusivamente pela função RolaDado() fornecida no template.
- O programa deve ser encerrado quando o caractere F (de Finalizar) for lido pela entrada padrão.
- Modularização e organização são fundamentais na construção do código. Você é obrigado a implementar os TADs solicitados via o template dos .h. Todavia, isso não impede a criação de outros.
- Todos os trabalhos serão corrigidos utilizando o sistema operacional Linux. Garanta que seu código funcione nos computadores do Labgrad. Qualquer discrepância de resultado na correção por conta de Sistema Operacional, os computadores do Labgrad serão utilizados para solucionar o problema.
- Todos os TADs implementados no trabalho devem ser opacos/encapsulados
- Possíveis problemas na descrição do trabalho serão solucionados e comunicados via Discord/Classroom. **Don't Panic!**

Considerações finais

Esse documento descreve de maneira geral as regras de implementação do trabalho. É de responsabilidade do aluno garantir que o programa funcione de maneira correta e amigável com o usuário. Qualquer alteração nas regras do trabalho será comunicada em sala e no classroom. É responsabilidade do aluno frequentar as aulas e se manter atualizado em

relação às especificações do trabalho. Caso seja notada qualquer tipo de inconsistência nos arquivos de testes disponibilizados, comunique imediatamente ao professor para que ela seja corrigida e reenviada para os alunos.